

# Design and Implementation of a Restaurant Order Management System Using JavaFX

Justin Mabry  
College of Arts & Sciences  
Indiana University - Bloomington  
Bloomington, USA  
jlmabry@iu.edu

Aden Yu  
Luddy School of Informatics, Computing, and Engineering  
Indiana University Bloomington  
Bloomington, USA  
adenyu@iu.edu

**Abstract**—This project implements a modular restaurant ordering system using Java, JavaFX, and four core object-oriented design patterns: Factory, Strategy, Observer, and Iterator. The system allows users to create orders, add menu items, select payment methods, and update order status through a graphical user interface. This report describes the system’s features, implementation details, challenges, team contributions, and the development timeline.

**Keywords**—component, formatting, style, styling, insert (key words)

## I. INTRODUCTION

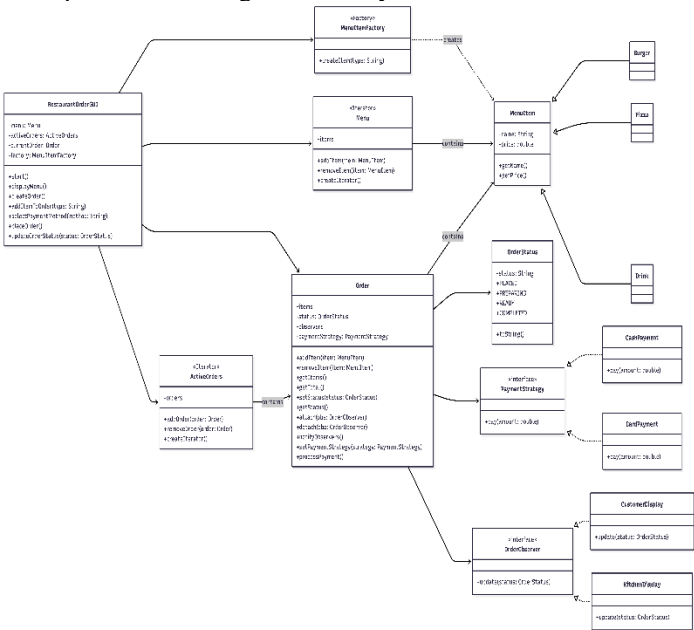
The goal of this project was to design and implement a restaurant ordering system that demonstrates the use of multiple design patterns within a cohesive software environment. The system includes menu item creation, order management, payment processing, and real-time status updates. A JavaFX GUI provides an interactive interface for users to place and manage orders.

## II. SYSTEM OVERVIEW

The implemented system consists of the following main features:

- **Menu Item Selection:** Users can choose from pizza, burger, and drink options displayed in a ListView. Each item includes a name and price.
- **Order Creation:** Items can be added to the current order, and the order contents are displayed in a text area.
- **Payment Processing:** Users may select either cash or card payment, each implemented as a strategy.
- **Order Status Updates:** The GUI acts as an observer and updates the displayed status when the order changes from PLACED to PREPARING, READY, or COMPLETED.

- **Design Pattern Integration:** The system incorporates Factory, Strategy, Observer, and Iterator patterns as required.
- A simplified UML diagram of the system is shown below.



## III. IMPLEMENTATION DETAILS

### A. Factory Pattern

The factory pattern is used to create menu items. The MenuItemFactory class contains a createItem(String type) method that returns instances of Pizza, Burger, or Drink. This allows the GUI to request items by name without knowing the underlying class structure.

Identify applicable funding agency here. If none, delete this text box.

### *B. Strategy Pattern*

Payment processing is implemented using the Strategy pattern. The `PaymentStrategy` interface defines a `pay(double amount)` method. `CashPayment` and `CardPayment` provide concrete implementations. The `Order` class stores a reference to a `PaymentStrategy` and delegates payment processing to the selected strategy.

### *C. Observer Pattern*

The Observer Pattern enables real-time updates to the GUI. The `Order` class maintains a list of observers and notifies them whenever the order status changes. The `RestaurantOrderGUI` class implements `OrderObserver` and updates the status label when notified.

### *D. Iterator Pattern*

An Iterator class (`MenuItemIterator`) is included to satisfy our project requirement. The iterator pattern is represented in the codebase and aligns with the UML, even if not directly used in the GUI. The version of the application before the GUI was implemented works from this pattern's method.

## IV. CHALLENGES ENCOUNTERED

### *A. JavaFX Configuration on Windows ARM*

A significant challenge involved configuring JavaFX on a windows ARM system. JavaFX does not provide native ARM binaries for my version of Windows, causing Maven dependency resolution failures.

### *B. Package and File Structure Alignment*

Java requires strict alignment between package declarations and directory structure. Early errors occurred when `RestaurantOrderGUI` declared a package that did not match its folder location. Removing the package declaration resolved this issue.

### *C. GUI integration with Backend Logic*

Ensuring the GUI interacted with the backend required strategic handling of menu item selection, string parsing for price display, and observer registration. Updating the `ListView` to include prices required adjusting the factory call to extract the item type.

## V. TEAM MEMBER CONTRIBUTIONS

### **JUSTIN MABRY**

- Implemented and refined the `RestaurantOrderGUI` class.
- Integrated the Observer pattern between `Order` and the GUI.
- Worked on order tracking, status updates, and GUI behavior.
- Performed testing and debugging of the overall system.

- Authored and edited the final project report.

### **Aden Yu**

- Created the initial UML system draft and updated it as the design evolved.
- Implemented the `MenuItem` hierarchy and `MenuItemFactory` (Factory Pattern).
- Implemented the `PaymentStrategy` interface and its concrete strategies (Strategy Pattern).
- Contributed to GUI components and layout.
- Set up the GitHub repository and managed version control.

## VI. PROJECT TIMELINE

The project followed the phases outlined in the proposal, with some natural overlap during our implementation and refinement.

### **Phase 1 (April 13-16):**

Finalized group and proposal, set up GitHub, drafted the initial UML, and mapped design patterns to use cases.

### **Phase 2 (April 17-23)**

Set up the Java project structure, implemented `MenuItem` classes and the Factory pattern, began JavaFX GUI layout, and defined the `Order` and payment class structure.

### **Phase 3 (April 24-27):**

Integrated Strategy and Observer patterns, connected the GUI to the underlying order and payment logic, evaluated order status updates, and refined UML diagrams.

### **Phase 4 (April 28-29):**

Finalized code, ensured the project was built and ran correctly, prepared the final PDF report, and completed submission.

## VII. CONCLUSION

This project successfully realizes the original goal of building a Java-based restaurant order system that demonstrates key object-oriented design patterns in a realistic context. The Factory pattern centralizes menu item creation, the Strategy pattern enables flexible payment processing, the Observer pattern provides real-time order status updates to the GUI, and the Iterator pattern supports structured traversal of collections. The system's architecture is modular and extensible, making it straightforward to add new menu items, payment methods, or additional observers such as kitchen or customer displays.

The implementation process also reinforced practical lessons about Java project structure, GUI integration, and dependency management. Overall, the final system is consistent with the proposal's intent and provides a clear, working example of design patterns applied to an interactive restaurant ordering workflow.